

GO POS PLAYGROUND

Architecture Episodes

Delapan konsep yang mengubah aplikasi CRUD menjadi sistem yang tahan retry, concurrency, kegagalan parsial, dan perubahan data.

Repository Pattern · Database Transaction · Idempotency · Stock Reservation · Async Payment · Audit Log · Soft Delete · Refresh Token

Daftar Episode

01	Kenapa Repository Pattern?
02	Kenapa Database Transaction?
03	Kenapa Idempotency?
04	Kenapa Stock Reservation?
05	Kenapa Async Payment?
06	Kenapa Audit Log?
07	Kenapa Soft Delete?
08	Kenapa Refresh Token?

Cara membaca

Setiap episode dimulai dari masalah, dilanjutkan pola penyelesaian, lalu menunjukkan lokasi implementasinya di repository. Fokusnya bukan menghafal istilah, tetapi memahami alasan sebuah pola muncul.

EPISODE 01

Kenapa Repository Pattern?

Repository memisahkan aturan penyimpanan data dari aturan HTTP dan tampilan aplikasi.

Masalahnya

Tanpa repository, handler mudah berubah menjadi tempat segala hal: membaca JSON, memvalidasi input, menulis SQL, mengunci row, menghitung stok, dan membuat response. Kode menjadi sulit dibaca dan aturan database tersebar.

HTTP Request → Handler → Repository → PostgreSQL

Pattern-nya

Handler menangani bahasa HTTP. Repository menangani bahasa persistence: query, transaction, locking, serta pemetaan row database menjadi entity.

- Handler fokus pada request dan response.
- Repository mengumpulkan SQL serta invariannya.
- Test dapat memanggil repository langsung tanpa browser.

Lihat di repo: `backend/internal/handler/cooperative.go` dan `backend/internal/repository/cooperative_repository.go`. Struct utamanya adalah `CooperativeRepository`.

Repository menjawab: “Bagaimana domain mengambil dan menyimpan data tanpa membuat semua lapisan memahami SQL?”

EPISODE 02

Kenapa Database Transaction?

Satu proses bisnis sering terdiri dari banyak query, tetapi semuanya harus dianggap sebagai satu perubahan.

Contoh checkout

1. Membuat header transaksi.
2. Membuat baris barang.
3. Mengubah stok.
4. Mencatat stock movement.
5. Membuat piutang atau payment.

Kalau langkah keempat gagal setelah tiga langkah pertama tersimpan, database menjadi setengah benar. Transaction memberi dua hasil saja:

```
Semua berhasil → COMMIT  
Satu saja gagal → ROLLBACK semuanya
```

Transaction juga bekerja bersama `SELECT ... FOR UPDATE`. Dua kasir yang menjual barang terakhir tidak boleh sama-sama membaca stok lama dan sama-sama berhasil.

Lihat di repo: fungsi `CreateTransaction` di `backend/internal/repository/cooperative_repository.go`. Cari `Begin(ctx)`, `FOR UPDATE`, `Commit`, dan deferred `Rollback`.

Transaction menjaga perubahan lintas tabel tetap atomik: lengkap semuanya atau tidak terjadi sama sekali.

EPISODE 03

Kenapa Idempotency?

Jaringan tidak dapat menjamin response selalu sampai, jadi request yang sama kadang harus dikirim ulang.

```
Frontend — checkout —> Backend
Backend membuat transaksi
Frontend ← response putus
Frontend melakukan retry
```

Tanpa idempotency, retry bisa membuat dua transaksi dan mengurangi stok dua kali. Frontend mengirim **Idempotency-Key**; backend menyimpan key dan hash payload.

- Key baru: proses checkout.
- Key dan payload sama: kembalikan transaksi sebelumnya.
- Key sama, payload berbeda: tolak sebagai conflict.
- Retry paralel: PostgreSQL advisory lock memastikan satu pembuat.

Lihat di repo: header dibaca di `backend/internal/handler/cooperative.go`. Replay detection dan `pg_advisory_xact_lock` ada di `cooperative_repository.go`. Skenario retry berada di integration test.

Idempotency menjawab: “Kalau request terkirim dua kali, bagaimana efek bisnisnya tetap hanya sekali?”

EPISODE 04

Kenapa Stock Reservation?

Payment belum berhasil, tetapi stok yang sedang dibayar juga tidak boleh dijual ke orang lain.

Stok fisik	10
Reserved	3
Tersedia	7

Reservation bukan pengurangan stok final. Ia adalah klaim sementara dengan masa berlaku.

PENDING → ACTIVE reservation
PAID → FULFILLED + kurangi stok
FAILED → RELEASED
EXPIRED → EXPIRED

Query ketersediaan menghitung stok fisik dikurangi semua reservation aktif yang belum kedaluwarsa. Dengan begitu aplikasi mencegah overselling tanpa menganggap payment pending sudah lunas.

Lihat di repo: pembuatan `stock_reservations` dan perhitungan reserved stock di `cooperative_repository.go`; finalisasi dan pelepasannya di `payment_repository.go`.

Reservation menjembatani perbedaan waktu antara “barang dipilih” dan “pembayaran sudah pasti”.

EPISODE 05

Kenapa Async Payment?

Hasil pembayaran digital tidak selalu diketahui pada saat transaksi dibuat.

Checkout → PENDING → polling/callback → PAID | FAILED | EXPIRED

Pada pembayaran tunai, kasir mengetahui hasilnya langsung. Pada pembayaran asynchronous, sistem menunggu aktivitas di waktu berbeda. Karena itu payment membutuhkan state machine.

Aturan terminal

Perpindahan yang sah adalah dari **PENDING** menuju salah satu state terminal. State terminal tidak boleh hidup kembali. Callback **PAID** setelah expiry harus ditolak, dan callback sama yang dikirim ulang harus idempotent.

Go POS hanya menyimulasikan pola ini. Tidak ada QRIS nyata, kartu, payment gateway, atau perpindahan uang.

Lihat di repo: lifecycle di `backend/internal/repository/payment_repository.go`, endpoint `status/simulator/cancel` di handler, serta countdown dan polling di `frontend/composables/useTransactions.ts` dan `frontend/utls/payment.ts`.

Async payment memisahkan waktu pembuatan transaksi dari waktu kepastian hasil pembayaran.

EPISODE 06

Kenapa Audit Log?

Data akhir memberi tahu apa kondisinya sekarang; audit log memberi tahu bagaimana kondisi itu terjadi.

Audit log membantu menjawab siapa melakukan apa, kapan, lewat endpoint mana, terhadap entity apa, dan apakah request berhasil.

```
Admin A • UPDATE • item 42  
PUT /items/42 • HTTP 200  
2026-08-05 14:30
```

Audit log berbeda dari application log. Application log membantu developer mendiagnosis sistem; audit log menjaga jejak tindakan bisnis pengguna.

Nama dan email disimpan sebagai snapshot. Jika akun pengguna kemudian dihapus, histori lama tetap dapat menjelaskan pelakunya dan foreign key tidak menghalangi penghapusan akun.

Lihat di repo: entity di `backend/internal/entity/audit_log.go`, persistence di `audit_repository.go`, middleware audit, dan migration identity snapshot di `database/postgres.go`.

Audit log adalah jejak bisnis yang tetap bermakna walaupun data utama berubah atau pengguna dihapus.

EPISODE 07

Kenapa Soft Delete?

Kadang “hapus” berarti menyembunyikan dari operasi aktif, bukan menghilangkan histori secara permanen.

```
Hard delete: DELETE FROM items WHERE id = 42;  
Soft delete: UPDATE items SET deleted_at = NOW() WHERE id = 42;
```

Query operasional memakai `deleted_at IS NULL`. Data terhapus masih bisa dipulihkan, transaksi lama tetap memiliki referensi, dan kesalahan operator lebih mudah diperbaiki.

Trade-off

- Setiap query aktif harus konsisten memfilter deleted row.
- Unique constraint sering perlu partial unique index.
- Restore harus memeriksa konflik kode atau SKU yang sudah dipakai kembali.
- Database tetap menyimpan data dan terus bertambah.

Lihat di repo: filter, delete, daftar terhapus, dan restore di `item_repository.go`, `suppliers_repository.go`, serta customer repository.

Soft delete menukar kesederhanaan hard delete dengan recovery, histori, dan keamanan operasional.

EPISODE 08

Kenapa Refresh Token?

Access token sebaiknya singkat untuk keamanan, tetapi pengguna tidak seharusnya login ulang setiap beberapa menit.

Login → access token pendek + refresh token panjang
Access token mendekati expiry → refresh → pasangan token baru

Refresh token disimpan pada cookie `HttpOnly`, sedangkan database hanya menyimpan hash-nya. Jika database bocor, raw token tidak langsung tersedia.

Rotation

1. Refresh token lama diverifikasi.
2. Token lama dicabut.
3. Session dan refresh token baru dibuat.
4. Token lama tidak dapat digunakan lagi.

Berbeda dari JWT panjang tanpa state, session refresh dapat dicabut ketika logout, akun dinonaktifkan, atau token digunakan ulang.

Lihat di repo: pembuatan dan hashing token di `backend/internal/auth/refresh.go`; create, rotate, dan revoke session di `backend/internal/repository/auth_repository.go`; orchestration HTTP di `backend/internal/handler/auth.go`.

Access token memberi akses singkat; refresh token menjaga kenyamanan login dengan session yang tetap bisa dikontrol server.

Peta Besar

Konsep	Masalah yang diselesaikan
Repository	SQL dan aturan persistence berceceran di handler.
Transaction	Perubahan lintas tabel tersimpan setengah jadi.
Idempotency	Retry jaringan menghasilkan efek bisnis ganda.
Stock Reservation	Overselling selama payment belum selesai.
Async Payment	Hasil pembayaran diketahui di waktu berbeda.
Audit Log	Tidak diketahui siapa menyebabkan perubahan data.
Soft Delete	Kesalahan hapus merusak histori dan sulit dipulihkan.
Refresh Token	Menyeimbangkan session nyaman dan access token pendek.

Judul commit yang disarankan

Payment lifecycle dan dokumentasi:

```
feat(payment): complete simulated async payment lifecycle
```

Migration audit log:

```
fix(database): harden audit log identity snapshot migration
```

Disusun dari implementasi aktual Go POS Playground. Tujuan project: portfolio dan playground pembelajaran, bukan sistem pembayaran produksi.